



WYDZIAŁ ELEKTRONIKI,
TELEKOMUNIKACJI
I INFORMATYKI

Dokumentacja Projektu grupowego

Dokumentacja techniczna projektu

Wydział Elektroniki, Telekomunikacji i Informatyki
Politechnika Gdańska

{wersja dokumentu wzorcowego: wersja 2/2023}

Nazwa i akronim projektu: Oprogramowanie do analizy obrazów błony kosmówkowo-omoczniowej (model CAM) zarodków jaj kurzych za pomocą sieci neuronowych	Zlecniodawca: Wydział Chemiczny PG, Katedra Chemii, Technologii i Biotechnologii Żywności	
Numer zlecenia: ID - 267	Kierownik projektu: Mateusz Nurczyński	Opiekun projektu: Szymon Olewniczak

Nazwa / kod dokumentu: Dokumentacja techniczna produktu – DTP	Nr wersji: 1.00	
Odpowiedzialny za dokument: Paweł Bogdanowicz Marek Szymański Michał Ganczarenko Mateusz Nurczyński	Data pierwszego sporządzenia: 11.06.2025	
	Data ostatniej aktualizacji: 15.06.2025	
	Semestr realizacji Projektu grupowego: 2	

Historia dokumentu

Wersja	Opis modyfikacji	Rozdział / strona	Autor modyfikacji	Data
1.00	Pierwsza wersja	całość	Paweł Bogdanowicz	11.06.2025
1.10	Uzupełniono podstawowe informacje o projekcie Dodano dokumentację techniczną wytworzoną podczas pierwszego semestru realizacji projektu	Rozdział 1 Rozdział 2.1.1	Paweł Bogdanowicz	11.06.2025
1.20	Opisano inne poza aplikacją do zbierania danych treningowych elementy infrastruktury procesu zbierania i przygotowania danych	Rozdział 2.1.2 Rozdział 2.1.3	Marek Szymański	13.06.2025
1.30	Opisano działanie modułu odpowiedzialnego za ocenę próbek w ramach aplikacji	Rozdział 2.4.4	Marek Szymański	14.06.2025
1.40	Udokumentowano wytworzone i przetestowane modele sieci neuronowych	Rozdział 2.3	Michał Ganczarenko	15.06.2025
1.50	Opisano infrastrukturę służącą do trenowania i testowania sieci neuronowych	Rozdział 2.2	Marek Szymański	15.06.2025
1.60	Uzupełniono opis biblioteki do filtrowania obrazów	Rozdział 2.2.2.2.1	Mateusz Nurczyński	15.06.2025
2.00	Udokumentowano funkcjonowanie aplikacji służącej do korzystania z wytrenowanych modeli	Rozdział 2.4	Paweł Bogdanowicz	15.06.2025

Spis treści

1	Wprowadzenie - o dokumencie	2
1.1	Cel dokumentu.....	2
1.2	Zakres dokumentu	2
1.3	Odbiorcy	3

1.4 Terminologia	3
2 Dokumentacja techniczna projektu.....	3
2.1 Infrastruktura do zbierania i wstępne przetwarzania danych treningowych	3
2.1.1 Aplikacja do zbierania danych treningowych.....	3
2.1.1.1 Ogólny opis aplikacji.....	3
2.1.1.2 Szczegółowy opis działania aplikacji	3
2.1.1.3 Opis techniczny aplikacji.....	3
2.1.2 Przycinanie zdjęcia	4
2.1.3 Wybór fragmentów zdjęcia zawierających badaną tkankę	4
2.2 Infrastruktura do treningu i testowania sieci neuronowych.....	4
2.2.1 Ogólny opis.....	4
2.2.2 Dane wejściowe	5
2.2.2.1 Format danych	5
2.2.2.2 Wstępna obróbka danych wejściowych.....	5
2.2.2.2.1 Filtry	5
2.2.2.2.2 Transformacje.....	6
2.2.2.2.3 Augmentacja	7
2.2.2.2.4 Normalizacja.....	7
2.2.2.3 Ładowanie danych wejściowych.....	7
2.2.3 Trening.....	8
2.2.4 Ocena	8
2.2.4.1 Metryki	9
2.3 Sieć neuronowa oceniająca próbki tkanki CAM	9
2.3.1 Ogólny opis.....	9
2.3.2 Wypróbowane modele	10
2.3.2.1 Model klasyfikacyjny	10
2.3.2.2 Model regresyjny	11
2.3.2.3 Klasyfikacja i regresja w jednym modelu	12
2.3.2.4 Własny model konwolucyjny	14
2.3.3 Funkcje kosztu	15
2.3.4 Hiperparametry	15
2.3.5 ResNet.....	15
2.4 Aplikacja do używania modelu	15
2.4.1 Ogólny opis aplikacji	15
2.4.2 Szczegółowy opis techniczny strony serwerowej.....	16
2.4.2.1 Inicjalizacja konfiguracji aplikacji	16
2.4.2.2 Inicjalizacja aplikacji.....	16
2.4.2.3 Moduł odpowiadający za uwierzytelnienie	16
2.4.2.4 Moduł odpowiadający za akcje administratora	17
2.4.2.5 Moduł umożliwiający korzystanie z klasyfikatora	17
2.4.3 Szczegółowy opis techniczny widoku przeglądarkowego aplikacji.....	18
2.4.3.1 Organizacja tras	18
2.4.3.2 Uwierzytelnienie.....	18
2.4.3.3 Podstrony	18
2.4.4 Szczegółowy opis techniczny modułu oceniającego próbkę	19

1 Wprowadzenie - o dokumencie

1.1 Cel dokumentu

Celem dokumentu jest udokumentowanie informacji dotyczących produktu, jego cech funkcjonalnych, parametrów technicznych, schematów blokowych, oprogramowania, wyników działania, zdjęć produktu, pomiarów, testów oraz innych elementów wymaganych przez opiekuna i klienta.

1.2 Zakres dokumentu

Dokument będzie stanowił dokumentację projektu. Będą się tutaj znajdować informacje na temat zasad działania aplikacji do zbierania danych treningowych.

1.3 Odbiorcy

Katedra Architektury Systemów Komputerowych, zespół projektowy.

1.4 Terminologia

CAM - błona kosmówkowo-omocznioowa

CRUD - Aplikacja tworząca, edytująca, usuwająca i czytająca dane (*create, read, update, delete*)

MVC - (Model, View, Controller) - Model wzorzec architektoniczny służący do organizowania struktury aplikacji posiadających graficzne interfejsy użytkownika (<https://pl.wikipedia.org/wiki/Model-View-Controller>)

HTML - Język język znaczników stosowany do tworzenia dokumentów hipertekstowych. (<https://pl.wikipedia.org/wiki/HTML>)

Framework - szkielet do budowy aplikacji

ResNet – rezydualna sieć neuronowa

CORS - (Cross-Origin Resource Sharing) mechanizm umożliwiający bezpieczny dostęp do zasobów między różnymi źródłami

2 Dokumentacja techniczna projektu

2.1 Infrastruktura do zbierania i wstępne przetwarzania danych treningowych

2.1.1 Aplikacja do zbierania danych treningowych

2.1.1.1 Ogólny opis aplikacji

Aplikacja umożliwia gromadzenie danych treningowych na potrzeby modelu sztucznej inteligencji. Została opracowana w języku Python 3 z wykorzystaniem frameworka Flask, który ułatwia tworzenie interfejsów API. Obecnie jest dostępna pod adresem: <https://kask.eti.pg.edu.pl/cam>. Przesyłanie zdjęć tkanek CAM odbywa się za pośrednictwem formularza, który dodatkowo zbiera informacje dotyczące każdego obrazu. Zgromadzone dane są następnie zapisywane na serwerze uczelnianym.

2.1.1.2 Szczegółowy opis działania aplikacji

Aby być w stanie wysłać zdjęcia należy się zalogować. Aplikacja w wypadku, w którym nie jesteśmy zalogowani automatycznie przekieruje nas do formularza do logowania. Możemy również od razu też wejść do tego formularza poprzez odpowiednie hiperłącze w sekcji nawigacyjnej strony. Sam formularz zawiera pole na wpisanie klucza uwierzytelniającego dostęp do naszego API. Po wpisaniu niepoprawnego klucza serwis nas o tym powiadomi, a jeśli podamy poprawny, to powrócimy do strony głównej.

Następnie użytkownik ma możliwość przeglądania istniejących danych lub dodania nowego zdjęcia. Aby dodać zdjęcie, należy kliknąć odpowiedni link w panelu nawigacyjnym, co spowoduje wyświetlenie formularza do przesyłania plików. Formularz umożliwia opcjonalne wprowadzenie wartości zmiennoprzecinkowych (Total area, Total length, Mean Thickness, Branching points) lub pozostawienie ich na domyślnym poziomie (-1). Użytkownik może także określić, czy dana tkanka nadaje się do dalszych badań oraz wprowadzić parametr skali, który musi być dodatnią liczbą całkowitą (domyślnie 0). Wybór zdjęcia tkanki CAM za pomocą selektora plików jest obowiązkowy. Po wysłaniu formularza użytkownik otrzyma informację o powodzeniu operacji.

Aby przejrzeć zgromadzone dane, należy wybrać opcję „View Data” z menu nawigacyjnego. Następnie zostanie wyświetlona lista zawierająca identyfikatory danych oraz daty ich dodania, wraz z odnośnikami do szczegółowych informacji. Kliknięcie w wybrany link spowoduje przejście do widoku szczegółowego danego zdjęcia tkanki CAM, gdzie dostępne będą wszystkie wprowadzone wcześniej wartości. Użytkownik będzie miał również możliwość usunięcia zdjęcia wraz z powiązanymi danymi poprzez opcję „Delete”.

2.1.1.3 Opis techniczny aplikacji

Aplikacja jest typowym systemem typu CRUD, jednak obecnie nie obsługuje edycji danych. Dane są przechowywane w bazie danych MongoDB. Aplikacja jest w pełni konfigurowalna za pomocą zmiennych środowiskowych. Użytkownik może w ten sposób określić host i port serwera, adres bazy danych MongoDB, katalog przechowywania zdjęć oraz klucz uwierzytelniający API.

Aplikacja została zaprojektowana zgodnie z wzorcem architektonicznym MVC. Składa się z warstw: widoków, kontrolerów, serwisów oraz repozytoriów. Kontrolery odpowiadają za odbieranie danych z zapytań, przekazywanie ich do odpowiednich serwisów, a następnie przesyłanie wyników do widoków, które zwracają użytkownikowi odpowiedź w formie kodu HTML. Serwisy wywołują repozytoria, które odpowiadają za pobieranie danych z bazy danych lub z plików.

Proces logowania opiera się na mechanizmie sesji dostarczonym przez framework Flask. W pierwszej kolejności następuje walidacja formularza, a następnie weryfikacja poprawności klucza uwierzytelniającego. Jeśli klucz jest prawidłowy, sesja użytkownika zostaje oznaczona jako uwierzytelniona, a użytkownik zostaje przekierowany na stronę główną.

2.1.2 Przycinanie zdjęcia

Jeszcze na wczesnym etapie prac okazało się, że bardziej wskazane niż praca nad pełnowymiarowym zdjęciem tkanki jest operowanie mniejszymi jego fragmentami. Dzięki przycięciu zdjęć do standaryzowanych wymiarów nie tylko łatwiejsza stała się współpraca z siecią neuronową, ze względu na niezależenie formatu danych wejściowych od wymiarów zdjęcia, ale przede wszystkim możliwe stało się wydawanie precyzyjnej oceny stanu zdrowia dla poszczególnych fragmentów tkanki, a nie tylko dla próbek - co zostało wskazane jako jedna z kluczowych funkcjonalności przez klienta. Ze względu na oczekiwany format danych wejściowych używanej pretrenowanej sieci neuronowej, na której bazuje nasz model rozmiar, do którego przycinane są zdjęcia to kwadrat o boku długości 224 pikseli. W przypadku gdy oryginalne zdjęcie nie posiada wymiarów będących równą wielokrotnością wymiarów pojedynczego fragmentu występuje konieczność dopełnienia niektórych fragmentów - konkretnie tych znajdujących się przy prawej i dolnej krawędzi - tak aby wszystkie fragmenty posiadały żądane rozmiary. Użytym kolorem dopełnienia jest kolor biały, wartość piksela 255. Sam podział dostarczonego przez użytkownika zdjęcia na fragmenty zrealizowano za pomocą skryptu w Pythonie i biblioteki Pillow. Pozwoliło to później zintegrować skrypt z aplikacją służącą do korzystania z wytrenowanego modelu – jak opisano w punkcie 2.4.4.

2.1.3 Wybór fragmentów zdjęcia zawierających badaną tkankę

Ponieważ zdjęcia próbek wykonywane na Wydziale Chemii nie ograniczają się zwykle jedynie do badanej tkanki, ale zawierają zwykle także elementy zbiornika zawierającego samą próbkę oraz tła, niektóre z fragmentów powstałych poprzez podział zdjęcia nie są przydatne do wykorzystania. Wystąpiła więc konieczność automatycznego odsiewania tychże fragmentów zanim jeszcze zostaną skierowane do modułu oceniającego próbkę. Wypróbowano dwie metody - jedną opartą o klasyczne algorytmy analizy obrazu oraz drugą wykorzystującą sieć neuronową. Ostatecznie bardziej skuteczna okazała się metoda druga, którą zaimplementowano poprzez dostrojenie lekko zmodyfikowanej rezyduacyjnej konwolucyjnej sieci neuronowej ResNet18, za której opracowaniem stali Kaiming He et al. z Microsoftu. Model wraz z pretrenowanymi wagami oraz możliwością edycji klasyfikatora dostarczyła biblioteka timm. Modyfikacje polegały na ograniczeniu sieci do jedynie 2 klas - "zawiera fragment tkanki" i "nie zawiera" - z oryginalnego tysiąca, co wykonano poprzez zmianę wymiarów ostatniej warstwy liniowej modelu, pełniącej rolę klasyfikatora, oraz dotrenowanie na ręcznie przygotowanym zbiorze danych zawierającym około 1000 fragmentów każdej z klas (zbiór podzielono na część służącą do treningu oraz walidacji w proporcjach 80 / 20). Finalnie wytrenowany i użyty model wykazał się dokładnością wynoszącą 99,3% na zbiorze testowym i sprawdzał się całkowicie zadowalająco według ludzkiej inspekcji wykonanej na około jednej trzeciej zbioru wszystkich klasyfikowanych fragmentów obu klas.

2.2 Infrastruktura do treningu i testowania sieci neuronowych

2.2.1 Ogólny opis

W ramach prac nad rozwijanymi modelami sieci neuronowych przygotowano wielo-elementową infrastrukturę służącą między innymi do wstępnej obróbki i ładowania danych, trenowania czy oceniania modelu. Całość prac prowadzono z wykorzystaniem języka Python w oparciu o framework pytorch służący do pracy z uczeniem maszynowym. Technologie te wybrano ze względu na rozległe możliwości, szerokie wsparcie przez inne technologie oraz wcześniejszą znajomość członków zespołu.

2.2.2 Dane wejściowe

W ramach współpracy w Wydziale Chemicznego Politechniki Gdańskiej przygotowano zbiór kilkuset zdjęć fragmentów tkanki CAM wraz z opisem ich właściwości wykonanym przez studentów Wydziału Chemicznego. Dane zebrane zostały z wykorzystaniem specjalnie do tego przygotowanej aplikacji opisanej w punkcie 2.1.1. Przygotowanie ich do współpracy z frameworkiem pytorch wymagało jednak jeszcze stworzenia odpowiedniego kodu umożliwiającego ich ładowanie do modelu sieci neuronowej. Ponadto, w celu zwiększenia efektywności uczenia sieci neuronowej, dane poddawane były wstępnej obróbce graficznej w celu uwypuklenia pożądanych cech, a także augmentacji.

2.2.2.1 Format danych

Pojedynczy punkt danych składa się ze zdjęcia (nazywanego dalej fragmentem zdjęcia lub fragmentem, ponieważ stanowi ono wycinek pełnowymiarowego zdjęcia próbki w formacie PNG, w 3-kanalowym standardzie kolorów RGC, uzyskany metodami opisanymi w punktach 2.1.2 i 2.1.3) o wymiarach 224 piksele na 224 piksele oraz słownika zawierającego opis danego zdjęcia (fragmentu zdjęcia).

Pełen opis fragmentu zawiera następującego pola:

- "id" – unikalny identyfikator (GUID) fragmentu nadawany przez aplikację do zbierania danych (patrz. Punkt 2.1.1.); 128-bitowa liczba zapisana szesnastkowo jako string
- "created_at" - data przesłania danego fragmentu do aplikacji służącego do zbierania danych; data i godzina w formacie yyyy-MM-ddTHH:mm:ss.SSSZ zapisana jako string
- "total_area" - całkowita powierzchnia widocznej na zdjęciu tkanki; nieujemna liczba zmiennoprzecinkowa; wartość specjalna -1.0 równoznaczna jest z brakiem wartości
- "total_length" - całkowita długość naczynek krwionośnych widocznej na zdjęciu tkanki; nieujemna liczba zmiennoprzecinkowa; wartość specjalna -1.0 równoznaczna jest z brakiem wartości
- "mean_thickness" - średnia grubość naczynek krwionośnych widocznej na zdjęciu tkanki; nieujemna liczba zmiennoprzecinkowa; wartość specjalna -1.0 równoznaczna jest z brakiem wartości
- "branching_points" - liczba rozgałęzień naczynek krwionośnych widocznej na zdjęciu tkanki; nieujemna liczba całkowita; wartość specjalna -1 równoznaczna jest z brakiem wartości
- "is_good" - binarna ocena stanu zdrowia widocznego na zdjęciu fragmentu tkanki; wartość boolowska
- "scale" - współczynnik skali w jakiej wykonano zdjęcie; nieujemna liczba całkowita
- "img" - nazwa pliku zawierającego opisywane zdjęcie

Można wyróżnić dwie grupy pól - pola opisujące bezpośrednio tkankę ("total_area", "total_length", "mean_thickness", "branching_points", "is_good") oraz pola niosące informacje natury organizacyjnej ("id", "created_at", "scale", "img").

W toku ciągłych prac i konsultacji z pracownikami Wydziału Chemicznego PG ustalono później, iż nie wszystkie z pierwotnie opracowanego na podstawie wstępnych konsultacji z WCh PG oraz przeglądu literatury naukowej przedmiotu zestawu pól będą przydatne w dążeniu do przyjętego celu. Wobec czego zdecydowano o wyeliminowaniu z dalszych prac następujących pól "total_area", "total_length", "mean_thickness", ograniczając opis tkanki do pól "branching_points" oraz "is_good", które stanowiły główny obiekt zainteresowania przeprowadzanej analizy.

Wszystkie zebrane zdjęcia przechowywano w jednym katalogu, a opisujące je słowniki w pojedynczym pliku w formacie JSON.

2.2.2.2 Wstępna obróbka danych wejściowych

W celu poprawienia efektywności uczenia opracowywanych sieci neuronowych zdecydowano się na wykonanie wstępnej obróbki obrazów podawanych do modelu. Głównym celem wstępnej obróbki było takie przetworzenie obrazu, aby uwypuklić te jego cechy, które powinny być, zgodnie z zebraną wiedzą ekspercką, najbardziej istotne podczas jego analizy – przede wszystkim sieć naczynek krwionośnych tkanki. Zastosowano zarówno tradycyjną filtrację obrazów jak i gotowe transformacje dostarczone przez bibliotekę torchvision. Ponadto w celu poprawy jakości zbioru treningowego wykonano augmentację wchodzących w jego skład zdjęć, mającą zapewnić bardziej różnorodny materiał do nauki dla sieci neuronowej.

2.2.2.2.1 Filtry

Na potrzeby projektu opracowano samodzielnie niewielką bibliotekę rozszerzającą funkcjonalność biblioteki *pillow* w dziedzinie filtracji obrazów. Rozwiązanie pozwala na skonfigurowanie potoku złożonego z wielu różnych filtrów i przepuszczenie przez nie zdjęcia. Dostępne są zarówno filtry macierzowe, jak i filtry operujące na pojedynczych pikselach.

Użycie biblioteki:

1. Tworzymy obiekt klasy *Filters*
2. Dodajemy filtry. Dostępne metody: *addFilter* (dodanie filtru na końcu potoku), *injectFilter* (dodanie filtru na wybranej pozycji potoku), *overwriteFilter* (nadpisanie filtru na określonej pozycji w potoku). Wszystkie metody przyjmują obiekty dziedziczące po abstrakcyjnej klasie *Filter*. Parametry filtru podaje się w konstruktorze. Dostępne klasy filtrów: *StandardPILFilter* (wrapper na gotowy filtr z biblioteki *pillow*), *MatrixFilter* (filtr macierzowy – jako parametr podajemy macierz), *PixelFilter* (jako parametr przyjmuje funkcję lambda, która zostanie wykonana na wszystkich pikselach)
3. Nakładamy wszystkie filtry w kolejności wywołując metodę *applyFilters*, która przyjmuje i zwraca zdjęcie w postaci obiektu klasy *Image*. W konstruktorze tego obiektu podajemy ścieżkę do pliku ze zdjęciem. Klasa posiada metodę *getTensor*, która pozwala uzyskać zdjęcie w formie tensora PyTorch.

Wykorzystane podczas prac filtry:

- Filtr konturowy biblioteki *pillow* *PIL.ImageFilter.CONTOUR*
- Filtr wyostrający biblioteki *pillow* *PIL.ImageFilter.SHARPEN*
- Filtr wzmacniający krawędzie biblioteki *pillow* *PIL.ImageFilter.EDGE_ENHANCE*
- Filtr macierzowy w postaci

0	-1	0
-1	4	-1
0	-1	0
- Filtr macierzowy w postaci

-1	-1	-1
-1	8	-1
-1	-1	-1

2.2.2.2.2 Transformacje

Biblioteka *torchvision*, stanowiąca część frameworku *pytorch*, służy do pracy z obrazami w kontekście uczenia maszynowego. Jednym z jej modułów jest *torchvision.transforms*, który dostarcza gotowych narzędzi pozwalających na wykonywanie wielu często wykorzystywanych przekształceń obrazów - nazywanych przez bibliotekę transformacjami. Dostarczone transformacje można także łączyć w sekwencyjne potoki, które następnie są aplikowane na przekształcanym obrazie. Biblioteka pozwala wykonywać operacje zarówno na obrazach *PIL.Image* jak i tensorach, co pozwoliło połączyć ją w własnymi narzędziami do filtracji obrazów opisanymi w punkcie 2.2.2.2.1.

Wykorzystane podczas prac transformacje:

- *torchvision.transforms.Resize* - przeskalowuje obraz lub tensor do podanych wymiarów
 - *torchvision.transforms.Grayscale* - konwertuje obraz do skali szarości, zwracając jeden lub trzy identyczne kanały
 - *torchvision.transforms.ToTensor* - konwertuje obiekt *PIL.Image* do obiektu *torch.Tensor*
 - *torchvision.transforms.Compose* - pozwala utworzyć zestaw sekwencyjnie aplikowanych transformacji
 - * *torchvision.transforms.RandomRotation* - z zadaniem prawdopodobieństwem dokonuje obrotu podanego na wejście obrazu o kąt w zadanym przedziale, dopełniając puste piksele zadanym kolorem
 - * *torchvision.transforms.RandomHorizontalFlip* - z zadaniem prawdopodobieństwem dokonuje obrotu o 180 stopni w poziomie podanego na wejście obrazu
 - * *torchvision.transforms.RandomVerticalFlip* - z zadaniem prawdopodobieństwem dokonuje obrotu o 180 stopni w pionie podanego na wejście obrazu
 - * *torchvision.transforms.ColorJitter* - z losowym prawdopodobieństwem modyfikuje jasność, kontrast, nasycenie oraz kolor podanego na wejście obrazu w zadanych przedziałach
- Zastosowanie transformacji oznaczonych symbolem * zostanie omówione szczegółowo w punkcie 2.2.2.2.3, odpowiadają one za augmentację zbioru treningowego.

Na szczególną uwagę zasługuje transformacja odpowiedzialna za przekształcenie obrazu do skali szarości. Ponieważ charakterystyka kolorystyczna tkanki nie różni się w wyraźny sposób pomiędzy

tkankami zdrowymi, a uszkodzonymi stanowi ona w pewien sposób informację nadmiarową. Konwersja do skali szarości miała więc za zadanie zmniejszyć ilość informacji podawanych na wejście sieci neuronowej, a w konsekwencji ułatwić jej zrozumienie ich znaczenia. Przeprowadzone eksperymenty porównawcze wskazują na niewielką przewagę sieci uczonych na obrazach w skali szarości w porównaniu z uczonymi na obrazach w pełnym kolorze, jednak różnice te są na tyle niewielkie, że nie pozwalają jednoznacznie stwierdzić o wyższości tej metody.

Podczas treningu wykorzystywane było złożenie wszystkich wymienionych powyżej transformacji, z albo bez transformacji Grayscale, zaś podczas testowania modelu nie używano transformacji odpowiedzialnych za augmentację danych – oznaczonych tutaj symbolem *.

2.2.2.2.3 Augmentacja

W kontekście uczenia maszynowego augmentacja polega na sztucznym zwiększeniu zbioru treningowego poprzez wprowadzeniu niego lekko zmodyfikowanych kopii danych już się w nim znajdujących. Celem jest poprawienie skuteczności modelu poprzez zwiększenie ilości i różnorodności danych uczących. Augmentacja jest szczególnie przydatna w przypadku posiadania niewielkiego zbioru danych, co było problemem napotkanym podczas prac nad projektem.

Opisana w punkcie 2.2.2.2.2 biblioteka torchvision dostarcza gotowych narzędzi - w postaci transformacji - służących do przeprowadzania augmentacji zbiorów składających się z obrazów.

Zastosowane metody augmentacji:

- Losowy obrót obrazu o kąt z zadanego przedziału - torchvision.transforms.RandomRotation
- Losowy obrót obrazu w poziomie - torchvision.transforms.RandomHorizontalFlip
- Losowy obrót obrazu w pionie - torchvision.transforms.RandomVerticalFlip
- Losowa modyfikacja kolorystyki (jasność, kontrast, nasycenie, kolor) obrazu - torchvision.transforms.ColorJitter

W konwencji przyjętej przez framework pytorch transformacje i przekształcenia obrazu wykonywane doraźnie po jego załadowaniu. W przypadku transformacji działających w oparciu o pewien rozkład prawdopodobieństwa oznacza to, że wykonają się one, w trochę odmienny sposób niż poprzednio, za każdym razem gdy poddany im obraz jest kierowany do modelu sieci neuronowej. Dzięki temu zbiór danych, na których przebiega trening modelu, zostaje poszerzony i urozmaicony.

Choć możliwe jest wykonanie wielu innych operacji w ramach augmentacji to zdecydowano się zastosować jedynie te, które tworzą na wyjściu obrazy nieodbiegające zbyt od tych możliwych do uzyskania w sposób naturalny.

2.2.2.2.4 Normalizacja

Przeprowadzono zarówno normalizację podawanych do sieci neuronowej obrazów, jak i danych numerycznych zawartych w atrybucie "branching_points" (opisanym w punkcie 2.2.2.1).

Normalizację wartości pikseli od zakresu 0-255 do zakresu 0.0-1.0 wraz ze zmianą rodzaju danych przeprowadzono za pomocą dostarczonej przez bibliotekę torchvision transformacji torchvision.transforms.ToTensor (opisaną w punkcie 2.2.2.2.2).

Normalizację wartości numerycznych atrybutu "branching_points" przeprowadzono za pomocą normalizacji min-max z zakresu 0-200 do 0.0-1.0 z pomocą samodzielnie zaimplementowanej funkcji normalizacyjnej, ponieważ framework pytorch nie dostarczał gotowego rozwiązania.

Nie przeprowadzono normalizacji wartości atrybutu "is_good" ponieważ jest to atrybut binarny. Zamiast tego przeprowadzono konwersję zmiennej boolowskiej na zmienną numeryczną (True = 1, False = 0).

2.2.2.3 Ładowanie danych wejściowych

We frameworku pytorch przyjęto dwuetapową procedurę ładowania danych do modelu, realizowaną przez współpracę obiektów dwóch klas – torch.utils.data.Dataset odpowiedzialnej za wczytywanie i przetworzenie danych do żadanego formatu oraz torch.utils.data.DataLoader pozwalającej modelowi uzyskać dostęp do wczytanych danych oraz realizującej batching. W praktyce w większości przypadków niezbędne jest dostarczenie własnej klasy implementującej torch.utils.data.Dataset (nazywanej dalej datasetem), która pozwoli odpowiednio dostosować procedurę ładowania posiadanych danych, ale nie jest konieczne tworzenie nowych implementacji torch.utils.data.DataLoader ponieważ dostarczona przez gotową klasę funkcjonalność jest wystarczająco uniwersalna by pasować do większości przypadków. Utworzone obiekty klasy torch.utils.data.Dataset przekazuje się do obiektów klasy torch.utils.data.DataLoader, które następnie zarządzają dostępem do danych.

Przygotowano klasę CamDataset, dziedziczącą i implementującą interfejs torch.utils.data.DataLoader, która wczytuje posiadane dane w formacie opisanym w punkcie 2.2.2.1 i przekształca zgodnie z tym co napisano w punkcie 2.2.2.2. Zapewniono możliwość wczytywania danych

bezpośrednio z plików formatu CSV, JSON oraz za pośrednictwem obiektów DataFrame popularnej biblioteki pandas. Dla zapewnienia maksymalnej elastyczności wczytane informacje o poszczególnych zdjęciach przechowywane są wewnętrznie w obiekcie klasy DataFrame. Przygotowany dataset zapewnia także możliwość konfiguracji wykonywanych na wczytywanych zdjęciach przekształceń poprzez podanie do konstruktora odpowiedniego zestawu filtrów (patrz 2.2.2.2.1) lub transformacji (patrz 2.2.2.2.2). Klasa pozwala także na ustawienie typu zwracanych danych, włączenie lub wyłączenie normalizacji atrybutu "branching_points" oraz filtrowanie danych wedle atrybutu "is_good" - to ostatnie pozwala przykładowo trenować model analizujący naczynka krwionośne jedynie na nieuszkodzonych fragmentach tkanki.

Zdjęcia ładowane są do pamięci w sposób leniwy – dopiero wtedy, gdy zażąda tego połączony DataLoader – a następnie wykonywane są na nich uprzednio skonfigurowane operacje przetwarzania. W pierwszej kolejności obraz ładowany jest do obiektu PIL.Image.Image, na którym następnie aplikowane są filtry, a na końcu transformacje. Istnieje także możliwość nieskorzystania z filtrów czy transformacji, choć transformacje Resize i ToTensor są wymagane, aby sieć neuronowa mogła przyjąć obraz. Pozostałe atrybuty - "is_good", "scale" oraz "branching_points" - przed zwróceniem podlegają jedynie konwersji typów do odpowiedniego rodzaju tensora.

W trakcie prac eksperymentowano także z różnorodnymi wariantami datasetów - zarówno ograniczającymi używane dane (np. tylko fragmenty zawierające więcej niż 15 rozgałęzień) oraz stosującymi inny format danych wyjściowych (np. zamiast dokładnej liczby rozgałęzień naczynek krwionośnych danego fragmentu tkanki przypisywano na ich podstawie zdjęcie do jednej z kilku klas i zwracano indeks klasy).

Format zwracanych danych:

- Krotka danych wejściowych
 - image - obraz jako tensor o wymiarach 3 x 224 x 224 liczb zmiennoprzecinkowych po normalizacji
 - scale - parametr skali jako jednoelementowy tensor
- Krotka danych docelowych
 - binary_target – jednoelementowy tensor binarny zawierający informację o stanie fragmentu tkanki (1 = zdrowa, 0 = niezdrowa)
 - regression_target – jednoelementowy tensor liczb zmiennoprzecinkowych zawierający znormalizowaną (lub nie) liczbę rozgałęzień naczynek krwionośnych danego fragmentu tkanki

Dane zwracane są jako krotka dwóch dwuelementowych krotek, z których pierwsza zawiera dane wejściowe do uczonego modelu, a druga wzorcowe dane do jego oceny – tzw. "ground truth".

2.2.3 Trening

W konwencji frameworka pytorch przyjęto, że trening modelu następuje wewnątrz tzw. "pętli treningowej", w której zawsze wykonywany jest zestaw powtarzalnych czynności, zwykle w poniższej kolejności:

1. Dane wejściowe oraz testowe pobierane są za pośrednictwem obiektu torch.utils.data.DataLoader (jak opisano w 2.2.2.3)
2. Model dokonuje predykcji na podstawie danych wejściowych
3. Predykcja modelu porównywana jest z danymi testowymi (stanowiącymi wartość prawdziwą, docelową) i z pomocą funkcji kosztu obliczana jest strata (loss) modelu
4. Zerowany jest gradient optymalizatora
5. Wykonywana jest propagacja wsteczna i wyliczane są gradienty na bazie wyznaczonej straty modelu
6. Wykonywany jest krok optymalizatora, który aktualizuje wagi trenowanego modelu na podstawie gradientu straty

Trening zwykle wykonywany jest w epokach, podczas których uczona sieć trenuje na danych podawanych kolejno w losowych batchach ustalonej wielkości.

Dla zapewnienia maksymalnej elastyczności pętlę treningową dopasowaną do używanych modeli i formatu danych zaimplementowano jako parametryzowaną funkcję znajdującą się w osobnym module. Dzięki temu można w łatwy sposób zaimportować ją i wykorzystać podając model do trenowania, dataloader, funkcję kosztu i liczbę epok treningu.

2.2.4 Ocena

W konwencji frameworka pytorch przyjęto, że testowanie modelu następuje wewnątrz tzw. "pętli testowania", w której zawsze wykonywany jest zestaw powtarzalnych czynności, zwykle w poniższej kolejności:

1. Dane wejściowe oraz testowe pobierane są za pośrednictwem obiektu `torch.utils.data.DataLoader` (jak opisano w 2.2.2.3)
2. Model dokonuje predykcji na podstawie danych wejściowych
3. Predykcja modelu porównywana jest z danymi testowymi (stanowiącymi wartość prawdziwą, docelową) i z pomocą funkcji kosztu obliczana jest strata (loss) modelu
4. Na podstawie predykcji modelu oraz wartości docelowych obliczane są metryki mierzące zachowanie modelu

Analogicznie jak trening testowanie zwykle wykonywany jest w epokach, podczas których uczona sieć trenuje na danych podawanych kolejno w losowych batchach ustalonej wielkości.

Dla zapewnienia maksymalnej elastyczności pętlę treningową dopasowaną do używanych modeli i formatu danych zaimplementowano jako parametryzowaną funkcję znajdującą się w osobnym module. Dzięki temu można w łatwy sposób zaimportować ją i wykorzystać podając model do trenowania, dataloader, funkcję kosztu i zestaw metryk do obliczenia.

2.2.4.1 Metryki

Aby precyzyjnie mierzyć skuteczność działania opracowywanych modeli wykorzystano klasy służące do obliczania różnorodnych metryk jakości stosowanych w uczeniu maszynowym dostarczanych przez bibliotekę `torchmetrics` wchodzącą w skład frameworka `pytorch lightning`. Ponadto opracowano własne metryki wykorzystujące analogiczny model działania, aby pokryć potrzeby, których nie zaspokajały gotowe rozwiązania.

Wykorzystane metryki:

Zastosowanie	Metryka
Klasyfikacja binarna stanu zdrowia fragmentu tkanki	<code>torchmetrics.classification.BinaryAccuracy</code> <code>torchmetrics.classification.BinaryPrecision</code> <code>torchmetrics.classification.BinaryRecall</code> <code>BinaryMetrics*</code>
Regresja liczby rozgałęzień naczynek krwionośnych fragmentu tkanki	<code>torchmetrics.regression.MeanSquaredError</code> <code>torchmetrics.regression.MeanAbsoluteError</code> <code>torchmetrics.regression.R2Score</code> <code>AverageRelativeError*</code> <code>AverageError*</code>

Metryki oznaczone symbolem * zostały zaimplementowane własnoręcznie

Własnoręcznie zaimplementowane metryki:

- `BinaryMetrics` – klasa obliczająca wartości `TruePositive`, `TrueNegative`, `FalsePositive`, `FalseNegative`, będące podstawą wielu metod oceny klasyfikacji binarnej. O ile klasy dostępne w ramach `torchmetrics` również dokonują tych samych obliczeń to ich wewnętrzne stany nie są dostępne na zewnątrz.
- `AverageRelativeError` – klasa obliczająca średni błąd względny dla wartości numerycznych
- `AverageError` – klasa obliczająca średni błąd bezwzględny dla wartości numerycznych

Ponadto opracowano klasę `CamMetricCollector` służącą do agregowania wielu metryk, która pozwala w wygodny sposób korzystać z większego zestawu metryk, bez konieczności indywidualnego aktualizowania czy zarządzania każdą z nich.

2.3 Sieć neuronowa oceniająca próbki tkanki CAM

2.3.1 Ogólny opis

W systemie zaprojektowano zestaw sieci neuronowych, z których każda pełni konkretną rolę w analizie obrazu fragmentów błony kosmówkowo-omoczniowej (CAM). Główne funkcje to:

- Weryfikacja obecności tkanki w obrazie (klasyfikator binarny)
- Ocena stanu zdrowia i tym samym przydatności próbki do dalszych badań (klasyfikacja binarna)
- Szacowanie liczby rozgałęzień naczyń krwionośnych (regresja)

Sieci te przetwarzają obrazy o wymiarach 224x224 piksele i mogą wykorzystywać zarówno wybrane konwolucyjne (ResNet, Unet) ekstraktory cech (`torch.nn.Module`), jak i modele pretrenowane z biblioteki `timm`. Sieci były trenowane i testowane z wykorzystaniem platformy CUDA, na kartach graficznych serii GTX 1060.

2.3.2 Wypróbowane modele

W toku realizacji projektu wypróbowano szereg architektur sieci neuronowych, które różniły się nie tylko strukturą wewnętrzną, ale również podejściem do problemu — niektóre skupiały się wyłącznie na klasyfikacji, inne na regresji, a część próbowała rozwiązać oba zadania jednocześnie. Przeprowadzono wiele eksperymentów porównujących skuteczność oraz stabilność działania modeli, zarówno pod kątem metryk ilościowych, jak i subiektywnej oceny przez ekspertów dziedzinowych.

2.3.2.1 Model klasyfikacyjny

CamNetClassifier to dedykowany model sieci neuronowej stworzony do realizacji pojedynczego zadania klasyfikacyjnego – określenia stanu zdrowia tkanki CAM (zdrowa / niezdrowa) na podstawie obrazu mikroskopowego pojedynczego fragmentu (patcha) o wymiarach 224×224 piksele. Model działa w trybie binarnej klasyfikacji, gdzie wynik wyjściowy interpretowany jest jako prawdopodobieństwo przynależności do klasy „zdrowa”.

Model ten opiera się na wykorzystaniu architektury ekstrakcji cech (feature extractor) dostępnej w bibliotece timm, która zapewnia dostęp do wielu pretrenowanych modeli głębokiego uczenia, takich jak ResNet, EfficientNet, MobileNet, ConvNeXt itd. Dzięki temu możliwa jest szybka adaptacja i transfer learning na stosunkowo niewielkich zbiorach danych.

Struktura modelu

Model został zaimplementowany jako podklasa torch.nn.Module. Posiada dwie główne części:

- Ekstraktor cech (feature extractor):
 - self.feature_extractor = timm.create_model(model_name, pretrained=True, num_classes=0)
 - model_name to parametr wejściowy pozwalający wybrać model bazowy np. 'resnet18'
 - num_classes=0 usuwa klasyfikator z końca modelu
 - uzyskujemy wektor cech o wymiarze feature_dim (różny w zależności od architektury)
- Klasyfikator binarny:
 - self.classifier = torch.nn.Sequential(
torch.nn.Linear(feature_dim + num_aux_inputs, FEATURES),
torch.nn.ReLU(),
torch.nn.Dropout(DROPOUT),
torch.nn.Linear(FEATURES, 1),
)
 - num_aux_inputs odpowiada liczbie dodatkowych danych wejściowych (np. Skala obrazu)
 - FEATURES i DROPOUT są ustalone globalnie w config.py (np. 256, 0.25)
 - ostatnia warstwa Linear zwraca pojedynczą wartość → prawdopodobieństwo bycia zdrowym

Dane wejściowe

Model oczekuje dwuelementowej krotki jako danych wejściowych:

- inputs = (image_tensor, scale)
- image_tensor: tensor o wymiarach (N, 3, 224, 224) – obraz w formacie RGB, gdzie N określa batch_size
- scale – skala w formacie zmiennoprzecinkowym

Dane wyjściowe

Zwracany jest tensor o wymiarze (N, 1), który zawiera prawdopodobieństwo przynależności do klasy „zdrowa” po przejściu przez funkcję aktywacji Sigmoid (zaimplementowaną poza klasą modelu, np. w funkcji oceniającej). Próg klasyfikacyjny może być konfigurowany (domyślnie: 0.5).

Przykładowy przepływ danych

Obraz trafia do modelu:

- `image_features = self.feature_extractor(image)`

Dane są łączone:

- `all_features = torch.cat([image_features, aux_data], dim=1)`

Wynik klasyfikacji:

- `output = self.classifier(all_features) # → (N, 1)`

Proces trenowania

Do trenowania modelu używana jest funkcja strat `BCEWithLogitsLoss()` – łącząca obliczenie funkcji kosztu z Sigmoid. W połączeniu z optymalizatorem Adam i technikami augmentacji (flipy, rotacje, color jitter), model uzyskiwał stabilne wyniki już po #TODO iteracjach.

Zastosowanie w systemie

W architekturze serwerowej opisanej w punkcie 2.4.4 model `CamNetClassifier` jest drugim krokiem w łańcuchu przetwarzania – po modelu `ResNet18` odrzucającym nieistotne fragmenty. Wyniki klasyfikatora są wykorzystywane do:

- wizualnej prezentacji (fragmenty zdrowe → kolor zielony, chore → czerwony)
- filtracji próbek pod dalszą analizę (np. regresję liczby rozgałęzień)

2.3.2.2 Model regresyjny

Cel modelu

`CamNetRegressor` został zaprojektowany jako specjalizowana sieć neuronowa do regresji liczby rozgałęzień naczyń krwionośnych w zdrowych fragmentach tkanki CAM. Jest wykorzystywany wyłącznie dla próbek wcześniej sklasyfikowanych jako zdrowe – decyzja ta zapada na wcześniejszym etapie potoku (klasyfikator stanu zdrowia).

Architektura modelu

Model ma niemal identyczną strukturę bazową jak `CamNetClassifier`, jednak zamiast klasyfikatora binarnego, posiada tylko głowicę regresyjną.

Struktura wygląda następująco:

- `self.regressor = torch.nn.Sequential(
 torch.nn.Linear(feature_dim + num_aux_inputs, FEATURES),
 torch.nn.ReLU(),
 torch.nn.Dropout(DROPOUT),
 torch.nn.Linear(FEATURES, 1),
)`
- `feature_dim` – liczba cech wyjściowych z ekstraktora (timm)
- `num_aux_inputs` – liczba dodatkowych zmiennych (np. skala zdjęcia)

Ostateczna warstwa to `Linear → 1`, co daje wynik w postaci jednej wartości typu float

Brak jest aktywacji typu Sigmoid, ponieważ model nie ogranicza wartości wyjściowej do zakresu `[0, 1]`, lecz zwraca liczby rzeczywiste (reprezentujące liczbę rozgałęzień).

Wejście i wyjście

- **Wejście:** (N, 3, 224, 224) + (N, num_aux_inputs)
- **Wyjście:** (N, 1) – estymowana liczba rozgałęzień dla każdego obrazu w batchu

Funkcja kosztu

Model trenowany jest z wykorzystaniem funkcji `MSELoss()` (ang. Mean Squared Error), która jest standardem w zadaniach regresyjnych:

- `loss = torch.nn.MSELoss()(predictions, targets)`

Zminimalizowanie błędu średniokwadratowego pozwala na uzyskanie stabilnej aproksymacji liczby rozgałęzień, nawet przy występowaniu pewnej zmienności pomiarów referencyjnych.

Wyzwania i specyfika regresji

- **Zakres wartości:** liczba rozgałęzień może sięgać od kilku do kilkudziesięciu, a nawet powyżej 100 w przypadku mocno unaczynionych tkanek.
- **Zróżnicowanie wizualne:** niektóre rozgałęzienia są bardzo subtelne lub nachodzą na siebie, co utrudnia precyzyjne oznaczenie referencyjne (ground truth).
- **Wpływ skali:** dlatego wartość skali (scale) jest często dołączana jako dodatkowe wejście (num_aux_inputs = 1).

Przykład działania

Dla pojedynczego batcha:

- `image_tensor = (1, 3, 224, 224)`
- `scale_tensor = (1, 1)`
- `output = model((image_tensor, scale_tensor))` # output → tensor([[17.64]]) # estymowana liczba rozgałęzień

Zastosowanie w potoku

Model `CamNetRegressor` jest ostatnim krokiem w analizie dla fragmentów uznanych za zdrowe. Na podstawie jego wyników można:

- **obliczyć sumę rozgałęzień w całej próbce (sumując wyniki z każdego fragmentu)**
- **wyznaczyć wskaźniki unaczynienia, przydatne w analizie rozwoju zarodka lub skuteczności terapii**

2.3.2.3 Klasyfikacja i regresja w jednym modelu

Opis ogólny

CamNet to uniwersalna, zintegrowana architektura sieci neuronowej, która łączy w sobie funkcjonalność klasyfikatora stanu zdrowia oraz regresora liczby rozgałęzień naczyń w jednym modelu. Opracowano ją z myślą o uproszczeniu pipeline'u przetwarzania obrazu i zwiększeniu spójności predykcji – umożliwiając jednocześnie określenie czy tkanka CAM jest zdrowa oraz jak bardzo jest unaczyniona.

Model ten wykorzystuje pojedynczy ekstraktor cech, wspólny dla obu zadań i rozdziela się na dwie niezależne głowice wyjściowe: klasyfikacyjną i regresyjną. Jego struktura i elastyczność pozwalają na łatwe przełączanie trybu działania (klasyfikacja, regresja, oba), co jest szczególnie przydatne na etapie eksperymentowania i porównywania jakości modeli.

Architektura

Model CamNet zawiera trzy główne komponenty:

- Ekstraktor cech (feature_extractor)
 - Wykorzystywany jest model z biblioteki timm (np. resnet18, efficientnet_b0)
 - Pretrenowane wagi, bez końcowego klasyfikatora
 - Zwraca reprezentację cechową o wymiarze feature_dim
- Głowica klasyfikacyjna (self.classifier)
 - `torch.nn.Sequential(
 Linear(feature_dim + num_aux_inputs, FEATURES),
 ReLU(),
 Dropout(DROPOUT),
 Linear(FEATURES, 1),
)`
 - Zwraca prawdopodobieństwo zdrowotności (output do Sigmoid)
- Głowica regresyjna (self.regressor)
 - Architektura identyczna jak powyżej, ale wyjście interpretowane jako liczba rozgałęzień
 - Nie zawiera aktywacji ograniczającej zakres

Obie głowice otrzymują identyczne dane wejściowe (cechy z ekstraktora + dane pomocnicze np. scale), ale działają niezależnie – można trenować je równolegle lub selektywnie.

Tryby działania

Dzięki parametrowi mode: Modes model może pracować w jednym z trzech trybów:

- Modes.CLASSIFIER – aktywna tylko głowica klasyfikacyjna
- Modes.REGRESSOR – aktywna tylko głowica regresyjna
- Modes.BOTH – aktywne obie głowice, zwracane są dwa wyniki

W `__init__` sprawdzany jest tryb działania, a w `forward()` odpowiednio dobierana logika zwracania wyników:

```
if mode == Modes.CLASSIFIER:

    return classifier_output, None

elif mode == Modes.REGRESSOR:

    return None, regressor_output

else:

    return classifier_output, regressor_output
```

Zalety integracji

- Oszczędność zasobów: jeden ekstraktor cech, brak potrzeby trzymania dwóch modeli w pamięci
- Spójność przetwarzania: ta sama reprezentacja użyta do klasyfikacji i regresji
- Elastyczność: model może być trenowany jako jeden lub dwa oddzielne komponenty
- Ułatwiona synchronizacja: regresja wykonywana tylko jeśli klasyfikator wskazuje, że fragment zawiera zdrową tkankę (można to wymusić po stronie aplikacji)

Uczenie i funkcje kosztu

Model obsługuje łączenie funkcji kosztu obu zadań:

- BCEWithLogitsLoss dla klasyfikacji
- MSELoss dla regresji

Przykład działania (mode = BOTH)

```
model = CamNet(model_name="resnet18", mode=Modes.BOTH)

img_tensor = torch.rand((8, 3, 224, 224))

scale_tensor = torch.rand((8, 1))

pred_class, pred_reg = model((img_tensor, scale_tensor))

print(pred_class.shape) # (8, 1)

print(pred_reg.shape) # (8, 1)
```

Uwagi praktyczne

- Choć architektura jest wspólna, możliwe jest różne przetwarzanie wejść dla obu zadań (np. różne filtry, augmentacje)
- Model jest bardziej złożony do trenowania – strata może konkurować (np. regresja poprawia się kosztem klasyfikacji)

2.3.2.4 Własny model konwolucyjny

Opis modelu

CamNetConv to własnoręcznie zaimplementowana architektura konwolucyjna, zbudowana od podstaw bez użycia gotowych ekstraktorów cech z bibliotek takich jak timm. Celem jego stworzenia była możliwość pełnej kontroli nad strukturą modelu i eksperymentowania z różnymi głębokościami, filtrami oraz funkcjami aktywacji w celu dopasowania modelu do charakterystyki danych CAM.

Model działał zarówno jako klasyfikator (np. zdrowa/niezdrowa tkanka), jak i jako regresor (liczba rozgałęzień), zależnie od zastosowania.

Architektura

Model składa się z następujących warstw:

- 7 warstw konwolucyjnych (Conv2d) z filtrami 3×3, przeplatanych aktywacją LeakyReLU
- warstwy MaxPool2d – redukcja wymiarów i ekstrakcja cech
- Końcowe spłaszczenie (Flatten)
- Dwie warstwy gęste (Linear):
 - 1323 → 256
 - 256 → 1

Właściwości modelu

- Wejście: RGB obraz 224×224 piksele
- Wyjście: pojedyncza wartość – interpretowana jako prawdopodobieństwo lub estymacja liczby rozgałęzień
- Rozmiar modelu: bardzo mały, szybki inference

Zalety i ograniczenia

Zalety:

- pełna kontrola nad architekturą i parametrami
- łatwa integracja i modyfikacja (np. testowanie nowych warstw)

Ograniczenia:

- niższa skuteczność niż modele z timm (np. brak normalizacji warstwowej, brak residuali)
- podatność na overfitting przy niewielkim zbiorze danych

Podsumowanie

Model CamNetConv był cennym komponentem do weryfikacji hipotez i porównań, jednak nie osiągnął wyników zbliżonych do modeli bazujących na pretrenowanych ekstraktorach. Jego główną wartością była możliwość pełnej modyfikowalności i testowania „czystych” pomysłów na architekturę.

2.3.3 Funkcje kosztu

Zależnie od modelu, zostały użyte różne funkcje kosztu zdefiniowane w osobnych plikach:

- MSE (Mean Squared Error) w przypadku regressora (ai/loss_fn/camloss_regressor.py)
- BCE (Binary Cross Entropy) w przypadku klasyfikatora binarnego (ai/loss_fn/camloss_classifier.py)
- Złożenie dwóch powyższych w przypadku modelu łączącego obydwa zadania (ai/loss_fn/camloss.py)

2.3.4 Hiperparametry

Parametr	Wartość	Uwagi
batch_size	32	Dobraną przez pryzmat wielkości datasetu
learning_rate	0.001	Standardowy
optimizer	Adam	momentum domyślny
num_epochs	10	trenowanie z wczesnym zatrzymaniem z uwagi na wielkość datasetu
dropout	0.2	warstwy klasyfikatora i regresora
augmentacje	flipy, rotacje, jitter	Szczegóły w 2.2.2.2
features	256	Liczba węzłów w wierzchnich warstwach Linear

2.3.5 ResNet

ResNet (ang. Residual Neural Network) to rodzina architektur sieci neuronowych opracowana oryginalnie przez Kaiminga He i jego zespół w 2015 roku, aby rozwiązać problem zanikania gradientów w sieciach głębokich. Architektura ta dobrze sprawdza się w m.in. zadaniach klasyfikacji obrazów i jest dostępna poprzez bibliotekę timm w postaci modeli ResNet18 i ResNet50 z pretrenowanymi wagami, które mogą zostać łatwo dostosowane do pracy z nowymi problemami. Z tego powodu sieci typu ResNet zostały wybrane jako podstawa do budowy modeli dokonujących analizy tkanki CAM.

2.4 Aplikacja do używania modelu

2.4.1 Ogólny opis aplikacji

Aplikacja umożliwia wykorzystanie modelu klasyfikującego tkanki CAM. Dostęp do serwisu wymaga uwierzytelnienia przy użyciu kont użytkowników, które tworzy wyłącznie administrator. Konto administratora jest generowane automatycznie podczas pierwszego uruchomienia aplikacji. Administrator ma możliwość zarządzania kontami użytkowników na dedykowanej podstronie - może je przeglądać, dodawać oraz usuwać.

Po zalogowaniu użytkownik uzyskuje dostęp do głównej podstrony - klasyfikatora tkanki CAM. Na tym ekranie należy najpierw wybrać zdjęcie, którego podgląd zostaje wyświetlony. Po uruchomieniu analizy obraz zostaje przesłany na serwer, gdzie podlega przetworzeniu przez model. Wynikiem są:

- procentowy udział zdrowych segmentów,
- łączna liczba wszystkich rozgałęzień,
- lista fragmentów o wymiarach 224x224 piksele, z których każdy zawiera informację o stanie tkanki (zdrowa/niezdrowa) oraz liczbie rozgałęzień.

Dane segmentów prezentowane są w formie tabeli nałożonej na zdjęcie, odwzorowującej ich pozycję i rozmiary. Każdy fragment jest oznaczony kolorem:

- zielonym – fragment z tkanką zdrową,
- czerwonym – fragment z tkanką niezdrową,
- szarym – brak tkanki jajka w obrębie fragmentu.

Po najechaniu kursorem myszy (lub dotknięciu palcem na urządzeniach mobilnych) wyświetla się okienko z dodatkowymi informacjami dotyczącymi danego fragmentu, wskazującymi jego stan zdrowia oraz liczbę rozgałęzień.

2.4.2 Szczegółowy opis techniczny strony serwerowej

Serwer aplikacji został napisany za pomocą języka python i bazuje na frameworku Flask. Umożliwia on tworzenie aplikacji internetowych w sposób elastyczny i rozszerzalny, przy minimalnym narzuceniu struktury projektu. Ponadto używana jest baza danych PostgreSQL za pomocą, której przetrzymywane są dane o kontaktach użytkowników. Aplikacja jest w pełni skonteneryzowana.

2.4.2.1 Inicjalizacja konfiguracji aplikacji

Klasa ConfigSettings odpowiada za wczytanie i walidację wszystkich niezbędnych ustawień aplikacji z pliku .env (jeśli dostępny) lub ze zmiennych środowiskowych. Kolejno pobierane są: tryb debugowania (DEBUG), adres i port serwera (SERVER_HOST, SERVER_PORT), klucz szyfrujący sesje (SECRET_KEY), URI bazy danych PostgreSQL (DATABASE_URL), a także dane administratora (ADMIN_USERNAME, ADMIN_PASSWORD). W przypadku braku lub nieprawidłowej wartości którejkolwiek zmiennej, konstruktor zgłasza wyjątek z informacją o brakującym parametrze. Flaga SQLALCHEMY_TRACK_MODIFICATIONS jest wyłączona, by ograniczyć zużycie pamięci. Po walidacji instancja Config udostępnia wszystkie ustawienia w całym projekcie.

2.4.2.2 Inicjalizacja aplikacji

Funkcja create_app zajmuje się inicjalizacją aplikacji. Przy wywołaniu tworzony jest obiekt Flask, któremu przekazujemy ścieżkę do katalogu statycznych plików Reacta, a następnie wczytujemy konfigurację z obiektu Config. W kolejnym kroku inicjalizowane są rozszerzenia: moduł bazy danych, menedżer logowania, mechanizm szyfrowania haseł oraz obsługa CORS.

Kolejnym etapem rejestracji aplikacji jest podłączenie tzw. blueprintów, odpowiadających za poszczególne domeny funkcjonalne. W obszarze uwierzytelniania ruch kierowany jest pod prefiks /cam/api/auth, zarządzanie kontami administratora - pod /cam/api/admin, natomiast logika klasyfikatora tkanki - pod /cam/api/classificator. Dzięki takiemu podziałowi kodu poszczególne moduły pozostają odseparowane i łatwe w utrzymaniu.

Aby umożliwić obsługę plików React, funkcja definiuje trasę serve_react_app, która przechwytuje żądania kierowane pod /cam oraz wszystkie jego podścieżki. Jeżeli zażądany zasób istnieje w katalogu build, zwracany jest bezpośrednio; w przeciwnym razie wysyłany jest plik index.html, co pozwala na prawidłowe działanie mechanizmu routingu po stronie klienta.

Ostatnim krokiem inicjalizacji jest wywołanie funkcji _initialize_database, która w kontekście aplikacji sprawdza istnienie tabel w bazie danych odpowiadających modelowi User. W razie potrzeby tworzy schemat bazy oraz jeśli nie istnieje dotychczas użytkownik administratora tworzy konto z danymi pobranymi ze zmiennych środowiskowych (ADMIN_USERNAME, ADMIN_PASSWORD).

2.4.2.3 Moduł odpowiadający za uwierzytelnienie

Moduł uwierzytelniania oparty na Flask Blueprint składa się z trzech głównych elementów: inicjalizacji blueprinta, definicji modelu użytkownika oraz obsługi tras odpowiadających za logowanie, wylogowanie i sprawdzanie statusu sesji.

Model użytkownika dziedziczy zarówno po db.Model, jak i po UserMixin z rozszerzenia Flask-Login. Klasa User definiuje cztery kolumny: id, username, password_hash oraz is_admin. Konstruktor przyjmuje nazwę użytkownika, hasło w postaci jawnej oraz opcjonalny parametr is_admin, automatycznie generując bezpieczny hash hasła przy użyciu bcrypt. Metoda check_password pozwala na weryfikację hasła, a get_id zwraca identyfikator użytkownika wymagany przez Flask-Login.

W ramach obsługi tras zdefiniowano trzy punkty końcowe pozwalające na korzystanie z systemu uwierzytelnienia:

- POST /cam/api/auth/login - oczekuje zapytania z polami username i password. Po weryfikacji poprawności danych i dostępie do rekordu użytkownika następuje wywołanie login_user, a serwer zwraca komunikat o powodzeniu operacji wraz z podstawowymi danymi użytkownika. W przypadku błędu walidacji lub nieprawidłowych poświadczeń zwracany jest stosowny komunikat i odpowiedni kod HTTP.
- POST /cam/api/auth/logout - dostępna wyłącznie dla zalogowanych użytkowników (oznakowana dekoratorem @login_required), wywołuje logout_user i potwierdza zakończenie sesji.
- GET /cam/api/auth/status - udostępnia informacje o aktualnym stanie sesji. Jeśli użytkownik jest uwierzytelniony, zwracane są jego dane (identyfikator, nazwa i uprawnienia administracyjne), w przeciwnym razie tylko informacja o tym, że brak aktywnej sesji.

Dodatkowo mechanizm ładowania użytkownika do kontekstu sesji realizowany jest przez funkcję oznaczoną jako @login_manager.user_loader, która na podstawie identyfikatora pobiera rekord z bazy danych.

2.4.2.4 Moduł odpowiadający za akcje administratora

Moduł odpowiadający za akcje administratora składa się z trzech elementów: deklaracji blueprints, dekoratora kontroli dostępu oraz zestawu tras realizujących operacje na kontach użytkowników.

W tym module zadeklarowany jest dekorator @admin_required. Jest to funkcja opakowująca logikę autoryzacji, wykorzystująca po kolei @login_required z Flask-Login, a następnie sprawdza, czy bieżący użytkownik ma nadane uprawnienia administratora. W przypadku braku takich uprawnień zwracany jest komunikat „Admin access required.” wraz z kodem HTTP 403 Forbidden.

Ten moduł definiuje trzy główne trasy dostępne tylko dla administratorów odpowiedzialne za operacje na kontach użytkowników:

- GET /api/admin/users - Zwraca listę wszystkich użytkowników w formacie JSON, zawierającą pola id, username i is_admin dla każdego konta.
- POST /api/admin/users - Pozwala na utworzenie nowego konta użytkownika. Metoda oczekuje żądania z polami username i password. W pierwszej kolejności weryfikowana jest obecność obu parametrów oraz unikalność nazwy użytkownika. W przypadku pomyślnej walidacji tworzony jest nowy obiekt User, dodawany do sesji bazy danych i zatwierdzany.
- DELETE /api/admin/users/<int:user_id> - Umożliwia usunięcie wskazanego użytkownika przez jego identyfikator.

2.4.2.5 Moduł umożliwiający korzystanie z klasyfikatora

Moduł umożliwiający korzystanie z klasyfikatora składa się z trzech elementów: deklaracji blueprints, deklaracji klasy transportowej oraz trasy umożliwiającej korzystanie z klasyfikatora.

W tym module definiowane są struktury danych wykorzystywane przez API klasyfikatora. Klasa SegmentData dziedziczy po BaseModel z Pydantic i opisuje pojedynczy fragment obrazu, zawierający liczbę punktów rozgałęzień oraz flagę stanu tkanki. Natomiast ClassificationResult agreguje wyniki całej analizy: sumę punktów rozgałęzień, odsetek zdrowych segmentów, wymiary fragmentów, ewentualnych fragmentów przyciętych na krawędziach, a także układ siatki i listę obiektów SegmentData.

Ekspozowany jest pojedynczy punkt końcowy /cam/api/classificator/, chroniony dekoratorem @login_required. Funkcja classify_image przyjmuje obraz wysłany jako multipart/form-data pod kluczem image, weryfikuje obecność i poprawność rozszerzenia pliku (dozwolone: PNG, JPG, JPEG) za pomocą funkcji allowed_file, a następnie odczytuje zawartość w bajtach. Następnie przekazuje dane obrazowe do warstwy serwisowej (opisanej w podpunkcie 2.4.4), oczekując zwrotu instancji DTO, którą serializuje do formatu JSON i odsyła klientowi z kodem HTTP 200. Błędy walidacji wejścia sygnalizowane są kodem 400, natomiast nieoczekiwane wyjątki podczas analizy kodem 500 oraz ogólnym komunikatem o błędzie.

2.4.3 Szczegółowy opis techniczny widoku przeglądarkowego aplikacji

Widok przeglądarkowy został stworzony przy użyciu frameworka React oraz języka TypeScript. Dodatkowo użyta została biblioteka tailwindcss w celu ułatwienia tworzenia responsywnej strony internetowej. Komunikacja z serwerem odbywa się za pomocą biblioteki axios.

2.4.3.1 Organizacja tras

Cały ruch jest przekierowywany na podstrony przez elementy Routes i Route z modułu react-router-dom. Zdefiniowano następujące ścieżki:

- Publiczne:
 - **/cam (index)** - Strona główna aplikacji
 - **/cam/login** - Podstrona będąca widokiem do logowania
- Wymaga zalogowania:
 - **/cam/classificator** - Podstrona będąca widokiem do korzystania z modelu
- Wymaga uprawnień administratora:
 - **/cam/admin/users** - Podstrona będąca widokiem do zarządzania użytkownikami
- Obsługa błędów:
 - **/cam/forbidden** - Podstrona przedstawiająca informację o braku uprawnień
 - **/cam/*** - Podstrona informująca o nieznalezieniu zasobu

2.4.3.2 Uwierzytelnienie

Moduł uwierzytelniania opiera się na kontekście React (AuthContext), który udostępnia informacje o aktualnie zalogowanym użytkowniku oraz funkcje logowania i wylogowania. AuthProvider wykonuje zapytanie do serwera pod ścieżkę `/cam/api/auth/status`, aby zweryfikować, czy użytkownik jest już zalogowany. W zależności od odpowiedzi serwera aktualizowany jest stan `currentUser`, co pozwala na jednolite zarządzanie stanem uwierzytelnienia w całej aplikacji.

Funkcja login wysyła dane logowania do `/cam/api/auth/login`, a w przypadku powodzenia zapisuje zwrócony obiekt użytkownika w stanie kontekstu. Analogicznie, logout wykonuje żądanie do `/cam/api/auth/logout`, czyści stan użytkownika i przekierowuje do strony głównej aplikacji. Dzięki wykorzystaniu kontekstu komponenty potomne mogą w prosty sposób uzyskać dostęp do bieżącego stanu uwierzytelnienia, jak również wywołać procedury logowania i wylogowania, zachowując przy tym spójność i centralizację logiki autoryzacyjnej.

2.4.3.3 Podstrony

Widok przeglądarkowy posiada następujące podstrony:

NotFoundPage

Jest to podstrona informująca o nieznalezieniu zasobu.

ForbiddenPage

Jest to podstrona przedstawiająca informację o braku uprawnień.

HomePage

Strona główna aplikacji pełni rolę punktu powitalnego i pierwszego kontaktu użytkownika z systemem. Po wejściu na tę podstronę użytkownik zostaje przywitany nagłówkiem "Welcome to CAM Classifier!".

W zależności od stanu uwierzytelnienia komponent dynamicznie dopasowuje treść komunikatu. W przypadku zalogowanego użytkownika prezentowany jest spersonalizowane powitanie z nazwą użytkownika oraz zachętą do rozpoczęcia pracy z klasyfikatorem. Użytkownicy niezalogowani otrzymują ogólny opis celu strony oraz sugestię zalogowania się, aby uzyskać pełny dostęp do funkcjonalności.

Całość oprawiona jest w estetyczny, responsywny układ z użyciem gradientowego tła, zaokrąglonych narożników i subtelnych cieni, co sprzyja przejrzystości interfejsu oraz komfortowi korzystania.

LoginPage

Podstrona logowania pełni funkcję centralnego punktu uwierzytelniania użytkownika w aplikacji

CAM Classifier. Formularz składa się z dwóch pól wymaganych: „Username” oraz „Password”, każde opatrzone odpowiednią etykietą i walidacją po stronie klienta. W przypadku błędnych danych uwierzytelniających pod nagłówkiem pojawia się komunikat o treści informującej o przyczynie niepowodzenia.

UserManagementPage

Moduł zarządzania użytkownikami stanowi panel administracyjny, w którym administrator może przeglądać istniejące konta oraz dodawać nowe. Strona otwiera się nagłówkiem „User Management”, wyraźnie sygnalizującym cel interfejsu, po czym inicjalizowany jest proces pobierania listy użytkowników z serwera. Całość umieszczono w responsywnym kontenerze o estetycznym, prostym układzie, zapewniając wygodny dostęp do wszystkich funkcji.

Pierwszym podkomponentem jest formularz tworzenia nowego użytkownika (CreateUserForm). Zawiera on pola „Username” oraz „Password” wraz z potrzebną walidacją, a także przycisk inicjujący zgłoszenie do serwera pod adres /cam/api/admin/users. W razie błędu wyświetlany jest komunikat z informacją zwrotną. Po pomyślnym utworzeniu konto zostaje dodane do głównej listy, co umożliwia administratorowi natychmiastowe zweryfikowanie działania operacji.

Drugi podkomponent to tabela użytkowników (UsersTable), wspomagana przez komponent wiersza (UserTableRow). Tabela prezentuje identyfikator, nazwę użytkownika oraz status uprawnień („Admin” lub „User”) w czytelnych kolumnach. W kolumnie akcji znajduje się przycisk „Delete”, który dla konta administratora jest dezaktywowany, aby zapobiec jego usunięciu. W przypadku potwierdzenia usunięcia odpowiednia funkcja komunikuje się z serwerem i usuwa dane z interfejsu, zachowując spójność widoku z rzeczywistym stanem bazy.

ClassifierPage

Podstrona klasyfikatora obrazów pełni rolę interfejsu do przesyłania i analizy zdjęć tkanek. Po załadowaniu komponent inicjalnie wyświetla kontrolki umożliwiające wybór pliku (tylko formaty PNG oraz JPEG) oraz przycisk „Analyze Image”. Wybrane zdjęcie zostaje zaprezentowane w formie podglądu, a w trakcie przetwarzania aplikacja sygnalizuje postęp za pomocą animowanego wskaźnika ładowania. W przypadku wystąpienia błędu (np. nieobsługiwany format lub problem sieciowy) użytkownik otrzymuje czytelny komunikat z czerwonym wyróżnieniem.

Gdy serwer zwróci wynik analizy, podgląd wzbogaca się o dodatkowe elementy. Nad obrazem wyświetlany jest nagłówek „Classification Result” wraz z informacją o wymiarach siatki segmentów (kolumn i wierszy). Poniżej dostępny jest przycisk pozwalający włączać lub wyłączać kolorowanie segmentów, co umożliwia łatwe porównanie surowego obrazu z wizualizacją oceny poszczególnych fragmentów.

Kolejnym elementem jest panel statystyk: procent zdrowych segmentów („Healthy Tiles”) oraz suma punktów rozgałęzień („Total Branching Points”). Główną sekcję stanowi komponent ImageDisplay, który otacza obraz responsywnym kontenerem, a następnie nakłada siatkę prostokątów - każdy reprezentowany przez komponent GridCell. Ten dynamicznie przypisuje klasę tła i obramowania w zależności od oceny (is_good): zielone dla segmentów pozytywnie zweryfikowanych, czerwone dla odrzuconych oraz szare dla nieweryfikowanych.

Wewnątrz każdego GridCell działa podkomponent SegmentTooltip, aktywowany podczas najechania kursorem. Wyświetla on szczegółowe dane segmentu: numer, stan („GOOD”, „BAD” lub „Unknown Value”) oraz liczbę punktów rozgałęzień.

2.4.4 Szczegółowy opis techniczny modułu oceniającego próbkę

Po stronie serwerowej aplikacji umieszczone wytrenowane modele dokonujące oceny próbki w ramach potoku, który dokonuje odpowiedniego przygotowania danych wejściowych oraz interpretacji wyników. Obecnie w procesie oceny próbki i podejmowania decyzji co do jego kroków uczestniczą trzy różne, niezależne sieci neuronowe, z których każda odpowiada za odrębny aspekt analizy.

Ponieważ na ten moment nie przewiduje się dużej częstotliwości zapytań do wytworzonej aplikacji poszczególne modele sieci neuronowych nie są przetrzymywane w pamięci w sposób ciągły, ale dla zachowania zasobów tworzone dopiero w momencie rozpoczęcia analizy zdjęcia, a po jej zakończeniu ponownie zwalniane. Modele odtwarzane są ze zlokalizowanych na serwerze aplikacji plików zawierających uprzednio wytrenowane wagi poszczególnych neuronów sieci neuronowej, co umożliwia odtworzenia jej zapisanego stanu.

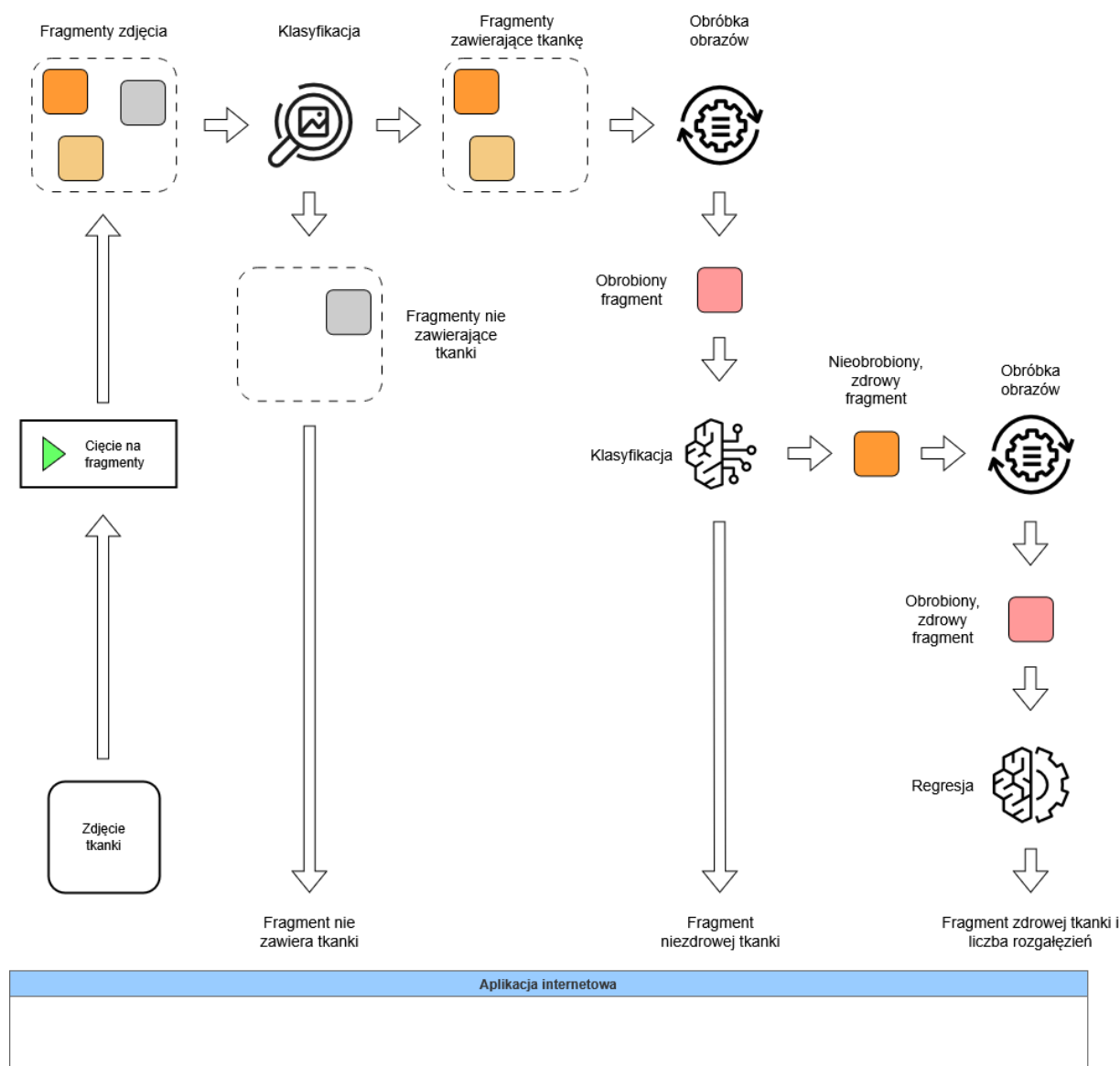


Diagram 1. Schemat dokonywania oceny zdjęcia tkanki przez sieć neuronową na serwerze aplikacji

Moduł oceniający tkankę rozpoczyna pracę w momencie otrzymania i skierowania do niego przez algorytm routingu serwera zdjęcia odebranego od użytkownika. W tej fazie wykonywana jest także walidacja odebranych danych. Następnie trafiające do modułu binarne dane zdjęcia zamieniane są na obiekt typu PIL.Image.Image z biblioteki pillow – format taki wybrano ze względu na szeroką dostępność mechanizmów obsługi w bibliotece pillow oraz dobrą współpracę z frameworkiem pytorch. Istotną operacją jest w tym momencie także konwersja zdjęcia do formatu RGB – wykonywana, aby zapewnić standaryzowaną liczbę kanałów, przekładającą się na rozmiar tensora, na wejściu sieci neuronowej. Mechanizmy zarówno wczytywania zdjęcia z binarnych danych jak i konwersji formatu zapewniane są przez bibliotekę pillow.

Jak wspomniano wcześniej przygotowane rozwiązanie nie operuje jedynie na całej próbce, ale dokonuje także odrębnej analizy jej poszczególnych fragmentów. Z tego powodu występuje konieczność podzielenia pełnowymiarowego zdjęcia próbki dostarczonego przez użytkownika na mniejsze fragmenty, które to zostaną poddane właściwej analizie. Proces ten zrealizowano w aplikacji z pomocą skryptu opisanego w punkcie 2.1.2 służącego do analogicznego dzielenia na fragmenty zdjęć używanych podczas trenowania używanych sieci neuronowych, jedynie przystosowanego do działania w odmiennych okolicznościach. Rozmiar docelowych fragmentów w dalszym ciągu wynosi 224 piksele na 224 piksele. Ponieważ jednak nie wszystkie fragmenty zawierają dane istotne z punktu widzenia badania – tzn. obraz tkanki CAM – a mogą zawierać także elementy niepożądane takie jak tło lub zbiornik na próbkę, niezbędne jest odfiltrowanie fragmentów niezdadnych do dalszej analizy. Zastosowano tutaj przygotowaną uprzednio sieć neuronową opisaną w punkcie 2.1.3 pełniącą rolę klasyfikatora binarnego. W aplikacji osadzono najlepszy z wytrenowanych modeli, cechujący się dokładnością rzędu 99%, wraz z

odpowiednią konfiguracją danych wejściowych i wyjściowych. Na podstawie decyzji klasyfikatora fragmenty zdjęcia są albo odrzucane – nie będą wtedy poddawane dalszej analizie, a użytkownik aplikacji zobaczy je jako wyszarzone – lub kierowane do dalszej analizy.

Kolejnym krokiem analizy, wykonywanym na fragmentach zakwalifikowanych przez klasyfikator fragmentów jako zawierające fragmenty tkanki jest sprawdzenie stanu zdrowia tkanki CAM. Jest to parametr binarny przyjmujący wartości “zdrowa” oraz “niezdrowa” i możliwy do ustalenia między innymi na podstawie obserwacji charakterystyki naczynek krwionośnych fragmentu tkanki – wpływ wywiera nań jednak także wiele innych czynników i procesów biochemicznych. Ocena atrybutu realizowana jest przez uprzednio przygotowaną sieć neuronową opisaną w punkcie 2.3.2.1 pełniącą rolę klasyfikatora binarnego. Należy tutaj zaznaczyć, iż jest to inna sieć neuronowa niż ta opisana w punkcie 2.1.3, która podejmuje decyzję o zaklasyfikowaniu fragmentu zdjęcia do dalszej analizy, choć również funkcjonuje ona jako klasyfikator binarny. Proces klasyfikacji poprzedzony jest przygotowaniem danych wejściowych polegającym na wstępnej obróbce analizowanego fragmentu zdjęcia z pomocą filtrów graficznych oraz transformacji dostarczanych przez bibliotekę torchvision - szczegółowe informacje o obróbce danych wejściowych zawarto w punkcie 2.2.2. Odpowiednio przetworzony fragment jest kierowany do klasyfikatora, na podstawie którego odpowiedzi podejmowana jest decyzja o skierowaniu fragmentów do dalszej analizy lub zakończeniu procesu. Fragmenty, na których przedstawiona tkanka zostanie uznana za zdrową kierowane są do kolejnego etapu, a te zawierające niezdrową tkankę nie są już analizowane dalej i użytkownik zobaczy je jako oznaczone kolorem czerwonym.

Ostatni etap analizy polega na wyznaczeniu liczby występujących we fragmencie rozgałęzień naczynek krwionośnych tkanki CAM. Ponieważ w tkankach niezdrowych naczynka te ulegają daleko idącym deformacjom - opuchnięciu, rozlaniu, pęknięciu - które w większości przypadków wykluczają możliwość wiarygodnego zliczenia rozgałęzień analiza podczas tego etapu wykonywana jest jedynie na tkankach zidentyfikowanych jako zdrowe. W przypadku opracowania alternatywnej metody pozwalającej na uwzględnienie tkanek niezdrowych proces zostanie odpowiednio zmodyfikowany. Liczba rozgałęzień jest parametrem nieujemnym numerycznym i do jej obliczenia wykorzystywana jest sieć neuronowa opisana w punkcie 2.3.2.2 wykonująca operacje regresji. Ponownie proces analizy poprzedzony jest odpowiednim przygotowaniem danych - należy zaznaczyć, że model zliczający rozgałęzienia otrzymuje osobną kopię analizowanego fragmentu, nie jest on więc zmodyfikowany przez obróbkę wykonaną na potrzeby działających wcześniej w procesie modeli, ponieważ każdy model może przyjmować dane przetworzone w odmienny sposób.

Ostatecznie moduł oceny próbki zwraca dla przetworzonego zdjęcia próbki wyniki analizy każdego pojedynczego fragmentu zdjęcia, zbiorcze informacje o całej próbce ustalone poprzez agregację wyników poszczególnych fragmentów oraz dodatkowe informacje niezwiązane z celem biznesowym, ale istotne dla warstwy prezentacji.

Pojedynczy fragment zdjęcia:

- Czy fragment zdjęcia zawiera fragment tkanki CAM? - jeżeli zdjęcie zawiera tkankę to pole informujące o jej stanie będzie zawierać wartość Prawda lub Fałsz, w przeciwnym razie pole to będzie puste wskazując, że fragment nie zawiera obrazu tkanki
- Czy widoczna na tym fragmencie zdjęcia tkanka CAM jest zdrowa? - wartość boolowska lub brak wartości, gdy zdjęcie nie zawiera tkanki
- Liczba zliczonych rozgałęzień fragmentu tkanki widocznego na zdjęciu. - nieujemna liczba całkowita (integer) lub brak wartości, gdy zdjęcie nie zawiera tkanki lub zawiera tkankę uszkodzoną

Cała próbka:

- Sumaryczna liczba zliczonych rozgałęzień naczynek krwionośnych w całej widocznym na pełnym, nieprzyciętym zdjęciu próbce. - nieujemna liczba całkowita (integer) stanowiąca sumę wszystkich zliczonych rozgałęzień wszystkich fragmentów zdjęcia
- Procent fragmentów zdjęcia zawierających fragment tkanki CAM, które zawierają zdrową tkankę - liczba zmiennoprzecinkowa pojedynczej dokładności (float) uzyskana poprzez ustalenie proporcji fragmentów tkanek zdrowych do wszystkich fragmentów zawierających tkankę - fragmenty niezawierające tkanki nie są uwzględniane w obliczeniach

Pozostałe informacje o przeprowadzonej analizie:

- Długość pojedynczego fragmentu zdjęcia
- Szerokość pojedynczego fragmentu zdjęcia
- Liczba fragmentów w pionie (liczba rzędów fragmentów)
- Liczba fragmentów w poziomie (liczba kolumn fragmentów)

- Długość fragmentów, które musiały zostać uzupełnione do pełnego rozmiaru
- Szerokość fragmentów, które musiały zostać uzupełnione do pełnego rozmiaru

Zadbano także o możliwość prostego rozszerzenia systemu o kolejne wytrenowane modele, poprzez opracowanie klasy integrującej dowolny, skonfigurowany model, właściwe dlań metody obróbki danych wejściowych (zarówno poprzez filtry jak i transformacje) oraz interpretację danych wyjściowych danego modelu. Dzięki temu, aby nowy model mógł współpracować z aplikacją konieczne jest jedynie dostarczenie jego definicji i zapisanego słownika wag, ustawienie ścieżki oraz skonfigurowanie danych wejściowych i wyjściowych. Możliwe jest także rozszerzanie procesu analizy o nowe etapy lub zmodyfikowanie aktualnie występujących.